

1 Time to Event and Censoring

Removing censored data will result in estimation uncertainty to be larger than it could be if we were to include the censored data. Under a frequentist approach, we would lose data points which will decrease the precision of our estimates. Removing censored data could also result in biased estimates if data have only been collected for a short time. These biased estimates would be obtained from those observations where the event of interest actually happened.

Random Right-censoring: All we know is that the patient did not die due to cancer until that moment.

Left-censoring: Happens when the event of interest occurred before you started observing someone, so you know the event time is earlier than a certain point, but you do not know exactly when.

Survival Function $S(t)$: Represents the probability that an individual or subject survives beyond a specific time t , or $P(T > t)$. This function is always monotonically decreasing, starting at 1 at $t = 0$ and approaching 0 as $t \rightarrow \infty$.

Cumulative Distribution Function (CDF) $F(t)$: Measures the probability that an event of interest has occurred at or before time t , defined as $F(t) = P(T \leq t)$. In survival terms, it is the complement of survival: $F(t) = 1 - S(t)$.

Probability Density Function (PDF) $f(t)$: Describes the unconditional rate at which events occur at a specific moment in time. Mathematically, it is the negative derivative of the survival function: $f(t) = \frac{d}{dt} F(t) = -S'(t)$.

Hazard Function $h(t)$: Describes the "instantaneous" risk or rate of an event happening at time t , given that the subject has survived up to that point. It is a conditional probability rate expressed as the ratio of the PDF to the Survival Function: $h(t) = \frac{f(t)}{S(t)}$.

1.1 Non-parametric—Surv func with Kaplan-Meier

We have no distributional assumption of survival function. **RESTRICTED mean:** it can be estimated as the area under an estimate of the survival function. **Quantiles:** they can be estimated by inverting an estimate of the survival function. In the `fit.km` output, if there is no upper bound in the CI, 50% of subjects were still alive by the end of study. See outputs of function.

```
fit_km <- survfit(formula = Surv(col1, col2) ~ 1, data = data)
tidy(fit_km) #gives 8 columns
quantile(fit_km, probs = 0.5, conf.int = FALSE) #median surv time
fit_km_plot <- autoplot(fit_km) #rmean-restricted mean in glance
glance(fit_km) |> mutate_if(is.numeric, round, 3)
```

1.2 Parametric—Surv func with Weibull

We need the corresponding parameters of the distribution. **Weibull:** monotonic. It either always increases, always decreases, or stays constant. If you believe your patients' risk of death only gets higher as time goes on (due to aging or disease progression), Weibull is a strong candidate.

Lognormal: non-monotonic (arc-shaped). It typically rises to a peak and then decreases toward zero for very long-term survivors. Common for cases where of patient survives the initial dangerous period(surgery), their long-term risk actually drops.

```
model <- survreg(formula = Surv(col1, col2) ~ 1, dist = "weibull",
  data = data) #dist="lognormal" if needed
intercept <- as.numeric(coef(model))
log_scale <- as.numeric(log(model$scale))
weibull_shape <- 1 / model$scale
weibull_scale <- exp(intercept)
median_weibull <- qweibull(p = 0.5, shape = weibull_shape, scale = weibull_scale)
mean_weibull <- weibull_scale * gamma(1+(1/weibull_shape))
```

1.3 Semi-Parametric-Cox Proportional Hazards Model

Do not assume any specific distribution for our response of interest. `fit_ph <- coxph(formula = Surv(col1, col2) ~ j_reg, data = data)` `tidy(fit_ph, exponentiate = TRUE, conf.int = TRUE)` `profile_data <- data.frame(ph.ecog = factor(2, levels = c(0, 1, 2)), meal.cal = 800)` `fit_specific <- survfit(fit_cox, newdata = profile_data)` `autoplot(fit_specific, conf.int = TRUE, surv.colour = "red")`

The treatment effect is significant because the p-value is far below $\alpha = 0.05$, so we reject H_0 . We have strong evidence against (H_0), so the treatment effect is statistically significant. **Interpretation:** Here, $(\exp(1.5721) = 4.8168)$, meaning that at any given time (t), the hazard of remission under the experimental treatment is 4.8168 times the hazard under placebo (assuming proportional hazards)

2 Piecewise Constant Regression

Goal is an accurate prediction in many complex frameworks (e.g., non-linear). fitting different **local OLS regressions**. The **step function** breaks the ranges of the regressor into intervals. In each interval, we adjust a constant. We obtain the average response in that interval. The regression model becomes $Y_i = \beta_0 + \beta_1 C_1(X_i) + \beta_2 C_2(X_i) + \dots + \beta_{a-1} C_{a-1}(X_i) + \beta_a C_a(X_i) + \varepsilon_i$

```
# Define the number of step functions (q + 1).
breaks_q <- 5
fat_content <- fat_content |> mutate(steps = cut(fat_content$week,
  breaks = breaks_q, right = FALSE))
levels(fat_content$steps)
model_steps <- lm(fat ~ steps, data = fat_content)
tidy(model_steps) |> mutate_if(is.numeric, round, 3)
glance(model_steps)
```

2.1 Non continuous Piecewise LR

So, for each interval beginning c_l , we have an additional intercept and an additional slope.

```
model_piecewise_linear <- lm(fat ~ week * steps, data = fat_content)
```

2.2 Continuous Piecewise Linear Regression

To fix the above matter, we can impose restrictions to make sure that the lines are connected to each other. Therefore, we need to introduce the concept of the hinge function.

```
levels(fat_content$steps)
knots <- c(7.8, 14.6, 21.4, 28.2)
model_piecewise_cont_linear <- lm(fat ~ week + I((week - knots[1])
  * (week >= knots[1])) + .., data = fat_content)
glance(model_piecewise_cont_linear)
```

3 k-nn Regression

K-nn fits better. Even if OLS predicts worse, can still be valuable for inference and decision justification. It provides interpretable effect estimates (with uncertainty) describing how each regressor is associated with the conditional mean monthly charge (holding other variables constant), which helps explain drivers and supports reporting or policy decisions.

```
# Use odd numbers so no mismatch on how the ties are handled.
my_knn <- knnreg(fat ~ week, data = fat_content, k = 7)
grid <- fat_content |> data_grid(week) |> add_predictions(my_knn)
```

4 (LOWESS) Regression

Identify the closest data points to the target x . `loess(ETA X, degree = 2, span = ...)` You try two settings for ... in span: • Model A: span = 0.20 • Model B: span = 0.80 — Model A is more flexible (wigglier) and may overfit noise; Model B is smoother and may miss real patterns

5 Quantile Regression

In many practical cases, we have inferential questions that are related to quantiles and not the mean. How does the alcohol price affect the weekly consumption of light drinkers, moderate drinkers, and heavy drinkers at given cut-off values of alcohol consumption? How does the alcohol price (X) affect the weekly consumption (Y) of light drinkers ($\tau = 0.25$), moderate drinkers ($\tau = 0.5$), and heavy drinkers ($\tau = 0.9$) in given cut-off values in weekly alcohol consumption? This prediction will not be a confidence interval since we are talking about probabilities (i.e., chances). Therefore, we can use "probability" instead of "confidence." Moreover, we would obtain prediction bands. MSE - i conditional mean. Pinball loss with $\tau = 0.5$ - i conditional median.

5.1 Parametric Quantile Regression

$Y_i = \beta_0(\tau) + \beta_1(\tau)X_{i,1} + \beta_2(\tau)X_{i,2} + \dots + \beta_k(\tau)X_{i,k} + \varepsilon_i(\tau)$ where $\beta_0(\tau)$ (for $j = 0, 1, 2, \dots, k$) means that the regression term has a quantile have a different regression function on the right-hand side of our modelling equation. Unlike OLS, where there is a single model fitting, any Quantile regression approach (either parametric or non-parametric) would require fitting as many models as quantiles we are interested in! OLS method minimizes the sum of squared residuals. Its goal is to estimate the conditional mean of the response variable. Quantile Regression method minimizes an asymmetric loss function (often called the "check function" or "tilted absolute value"). Its goal is to estimate a conditional quantile (like the 75th percentile).

— When $\tau = 0.25$, we consider overpredicting. $e_i = y_i - \hat{y}_i < 0$ WORSE than underpredicting $e_i = y_i - \hat{y}_i > 0$ Note how Steeper the penalty on fidelity graph (y-axis), when we overestimate (negative part of x-axis)

— When $\tau = 0.5$, we care equally about underpredicting and over-

predicting. This corresponds to the median value. — When $\tau = 0.75$, we consider underpredicting. $e_i = y_i - \hat{y}_i > 0$ WORSE than overpredicting $e_i = y_i - \hat{y}_i < 0$

```
fit_rq_teams <- rq(formula = runs ~ hits, data = teams, tau =
  c(0.25, 0.5, 0.75))
# pairwise bootstrapping method resamples with replacement bsmethod
param_QR_summary <- summary(fit_rq_teams, se = "boot", R = 1000,
  bsmethod = "xy")
param_QR_summary[3] #75 percentile
round(predict(fit_rq_teams, newdata = data.frame(hits = 1000)), 0)
#prediction
```

Inference: For those teams at the upper 75% threshold in runs, is this association is significant. So hits is statistically associated to the 75-quantile of runs. **Coef-Interpretation:** For each one hit increase, the 75-quantile of runs will increase by 0.5. **Prediction:** for any given team that scores 1000 hits in a future tournament, it is estimated to have a 50% (0.75-0.25*100%) chance to have between 452 and 557 runs. "Among patients arriving around now, about 9 out of 10 will be seen within 30 minutes, and about 1 out of 10 will wait longer."

5.2 Non-Parametric Quantile Regression

This class of Quantile regression implicates no model function specification. The core idea is to allow the model to capture the local behaviour of the data (i.e., capturing its local structure). Lambda decides variability. smaller lambda estimated functions will not be that smooth anymore, there will be more variability on the prediction.

```
rqss_model_0.25_lambda_100 <- rqss(runs ~ qss(hits, lambda =
  100), tau = 0.25, data = teams)
summary(rqss_model_0.25_lambda_100)
```

6 Missing Data

Identifying the type of missingness is critical because it determines the appropriate statistical approach. **Missing Completely At Random (MCAR):** Missingness is totally by chance and independent of any data. It is the ideal class because there is no pattern; removing these rows does not introduce bias, though it increases standard errors due to a smaller sample size. **Missing At Random (MAR):** Missingness depends on observed data available in the dataset. For example, a device might fail more often at night, and "time" is a recorded variable. Imputation techniques can use the observed data to fill these gaps. **Missing Not At Random (MNAR):** The most serious type, where missingness depends on unobservable quantities. If the variable causing the missingness is not recorded, the data is MNAR, and simple deletion will likely skew the analysis and introduce serious bias. **Simple Handling Techniques & Their Drawbacks** **Listwise Deletion:** Removing any row with missing data. While unbiased for MCAR, it leads to loss of power (higher standard errors) and can cause significant bias in MAR or MNAR scenarios. **Mean Imputation:** Filling NAs with the overall mean of the variable. ◦ Constraint: Only for continuous or count-type variables. ◦ Drawback: It artificially reduces the standard deviation/variance of the data and ignores relationships between variables. **Regression Imputation:** Uses an OLS model from complete observations to predict missing values. ◦ Drawback: It reinforces the data's correlation too perfectly, as all imputed points fall exactly on the regression line, which can "inline" the behavior artificially. **multiple imputation** is robust enough to work for both MAR and MNAR data. Create several completed datasets by filling in missing values with plausible draws.

```
md.pattern(data): Used for Exploratory Data Analysis (EDA) to
  visualize missingness patterns.
mice(data, m = 5, method = "pmm"): Generates m imputed datasets.
  with(mids, lm(y ~ x)): Performs analysis on all imputed datasets
  simultaneously.
pool(models): Combines multiple models into one using Rubins
  Rules.
summary(pooled_model): Provides the final estimates, standard
  errors, and p-values
```

